

What Is Memory Management?

Memory Management (MM) is the function of the Operating System that **controls and coordinates** the computer's **main memory (RAM)**. It keeps track of which parts of the memory are currently being used by which programs and which parts are free.

- **The Goal:** To efficiently and securely allocate portions (blocks) of RAM to various running programs (processes) to optimize overall system performance and enable **multitasking**.
- **The Crucial Role:** Since no process can ever run before a certain amount of memory is allocated to it, the MM is the gatekeeper that decides **who gets memory, when, and how much**.
- **Analogy:** Think of the OS memory manager as a **landlord** in a large apartment building (RAM). The landlord must:
 1. Keep a map of all occupied and vacant apartments.
 2. Assign an apartment to a new tenant (a running program).
 3. Ensure one tenant cannot break down the walls and interfere with the neighbor's apartment.
 4. Sometimes move tenants to a new apartment (relocate) if the building needs reorganization.



Five Requirements of Memory Management

To achieve efficiency, protection, and flexibility in a modern multitasking environment, the Memory Management system must satisfy five key requirements:

1. Relocation

The ability of a program to be loaded into any physical section of main memory and still execute correctly.

- **The Problem:** In a multitasking system, a program might be swapped out to disk and then swapped back into a *different* memory location later (to utilize available space). However, the program code is written with internal addresses (e.g., "jump to instruction 500"). If the program is moved, instruction 500 might now point to another program's memory!
- **The Solution:** The OS must find a way to map the program's internal addresses, called **Logical Addresses (or Virtual Addresses)**, to the actual memory locations, called **Physical Addresses**.
 - **Mechanism:** This is typically done dynamically (at execution time) using a hardware component called the **Memory Management Unit (MMU)**. When the CPU generates a logical address, the MMU uses registers (like a **Base Register**) or tables (like a **Page Table**) to translate it into the correct physical address instantly.
- **Key Concept:** Relocation ensures memory space can be efficiently managed without being fixed to specific programs.

2. Protection

The ability to prevent a process from reading or writing to a memory area that has not been explicitly allocated to it.

- **The Problem:** Without protection, a buggy or malicious process could accidentally or intentionally overwrite the code or data of another running process or even the Operating System itself, leading to system crashes or security breaches.
- **The Solution:** Every process should be protected against unwanted interference by other processes.
 - **Mechanism:** Protection is often tied directly to the relocation hardware. For example, using **Base and Limit Registers**: the MMU checks if the logical address is within the process's allocated range (between the Base and Limit). If it is outside this range, the hardware triggers a **memory access violation** (a type of interrupt).
- **Key Concept:** Protection isolates processes to maintain system stability and security.

3. Sharing

The ability to allow multiple processes to access the same portion of main memory in a controlled manner, without compromising the Protection requirement.

- **The Need:** If ten different programs (processes) are all using the same system library (e.g., a function to display a window), it is inefficient to load ten separate copies of that library into RAM.
- **The Solution:** The protection mechanism must be flexible enough to allow several processes to access the same portion of memory.
 - **Mechanism:** Sharing is achieved by setting up the address translation tables (like Page Tables) so that the logical addresses of two different processes map to the *same* physical memory address.
- **Key Concept:** Sharing improves memory utilization and reduces the overhead of loading common libraries.

4. Logical Organization

Memory management should support the way programs are naturally structured.

- **The Program View:** Most programs are organized into logical **modules** or **segments** (e.g., a main program, a stack, a set of subroutines, a data array). These modules often need different types of protection (e.g., the code module is "Execute Only," while the data module is "Read/Write").
- **The Requirement:** While main memory is organized as a single, linear (1-D) sequence of bytes or words, the OS and hardware must support this modular view to allow:
 - **Easier protection and sharing** (assigning permissions segment-by-segment).
 - **Independent loading** (only load the necessary segment when needed).
- **Mechanism: Segmentation** is the memory management technique designed to directly support this logical organization.

5. Physical Organization

Memory management must account for the two-tier structure of computer memory.

- **The Physical Reality:** The computer memory is organized as:
 - **Main Memory (RAM):** Provides **fast access** but is **volatile** (loses data when power is off) and **limited** in size.
 - **Secondary Memory (Disk/SSD):** Provides **long-term retention** (non-volatile) and is **vastly larger** but much **slower** to access.
- **The Requirement:** The OS memory manager must actively manage the flow of information between these two levels.
 - **Mechanism:** Techniques like **Swapping** and **Virtual Memory (Paging)** are used to manage this physical organization, allowing processes larger than physical RAM to run by moving currently unused blocks of data/code between RAM and the secondary memory.
- **Key Concept:** This enables the system to run many large programs concurrently, increasing the degree of **multiprogramming**.



Address Binding in Operating Systems

Address Binding is a core process in **Memory Management** that links the addresses used in a program (the logical addresses) to the actual hardware locations in the main memory (the physical addresses).

The Operating System (OS) performs this crucial function on behalf of an application so that instructions and data can be fetched and executed by the CPU.

Logical vs. Physical Address Space

To understand binding, you must know the difference between the two address types:

Address Type	Generated By	What it Represents	Key Feature
Logical Address (or Virtual Address)	The CPU (when executing a program)	The address as the program <i>sees</i> it, independent of where it actually sits in RAM.	Allows relocation and virtual memory .

Address Type	Generated By	What it Represents	Key Feature
Physical Address	The Memory Management Unit (MMU)	The actual, real location of the data or instruction in the main memory (RAM) .	Used by the memory hardware to access the physical chip.

The Binding Process is the translation of the Logical Address generated by the CPU into the Physical Address used by the memory hardware.



Types of Address Binding (When Binding Occurs)

The key difference between address binding types is the **point in time** when this translation (binding) takes place. This choice dictates the flexibility and complexity of the memory system.

1. Compile Time

- **When it Happens:** During the **compilation** of the source code into an executable program.
- **Mechanism:** If the programmer or compiler knows exactly where the program will be loaded in physical memory (e.g., always starting at address 1000), the compiler generates **absolute code**.
- **Result:** The Logical Address and the Physical Address are **identical** and **fixed** before the program ever runs.
- **Flexibility: Zero.** The program can only run if the exact starting location is available. This is extremely rigid and not used in modern multitasking systems.

2. Load Time

- **When it Happens:** When the program is **loaded** from secondary storage (disk) into main memory (RAM).
- **Mechanism:** If the starting location is unknown at compile time, the compiler generates **relocatable code** (addresses relative to the program's starting point, e.g., "300 bytes after the start"). The **Loader** then determines the actual starting address in RAM (the **Base Address**) and adds it to all the relative addresses to form the final physical addresses.
- **Result:** The Logical Addresses are still mapped to Physical Addresses, but the physical addresses are **fixed** for the duration of the program's execution.
- **Flexibility: Static.** The program can be loaded anywhere in memory, but once loaded, it cannot be moved (relocated) while running.

3. Execution Time (Run Time / Dynamic)

- **When it Happens:** During the **execution** of the program, every time an instruction or data is accessed.
- **Mechanism:** The CPU generates a Logical Address, and a hardware device called the **Memory Management Unit (MMU)** performs the translation to the Physical Address *on the fly*. This is usually done using a **Relocation Register** or **Page Tables**.
- **Result:** The binding is **dynamic**. The process can be **moved (relocated)** in memory during execution because the translation hardware (MMU) handles the change instantly.
- **Flexibility: Maximum.** This is the method used by **all modern, general-purpose Operating Systems** (like Windows, macOS, Linux) to support multitasking and virtual memory.



Advanced Memory Techniques

These techniques rely heavily on **Execution Time Address Binding** to manage memory efficiently.

Dynamic Loading

- **What it is:** A technique where routines (like a large function or module) are **not loaded** into memory until they are **actually called** during the program's execution.
- **Mechanism:** The main program is loaded, but the code for a specific routine (e.g., an error handler) stays on disk. When the program calls that routine, the OS loads it into an available memory space and updates the program's address tables.
- **Benefit:** Memory is only used for the components of the program that are truly needed at any given time, saving space.

Dynamic Linking

- **What it is:** A technique that postpones the linking of shared libraries (like `.dll` files in Windows or `.so` files in Linux) until **execution time**.
- **Mechanism:** Instead of including the actual code for a function (e.g., `printf()`) in the program's executable file, the executable contains only a small stub. When the function is called, the OS dynamically links to the shared copy of the library already loaded in memory (or loads it if needed).
- **Benefit:** **Saves disk space** (executables are smaller) and **saves RAM** (multiple programs share one copy of the library in physical memory).

Overlays

- **What it is:** A technique used in very old or embedded systems with extremely limited memory, where the total program size is **larger than the physical memory**.
- **Mechanism:** The program is manually divided into modules (**overlays**) that are loaded into the same physical memory space on an as-needed basis. Only the main core and one overlay segment are in memory at any time.
- **Benefit:** Allows large programs to run on small machines.
- **Status:** Largely **obsolete** in general-purpose OSs, replaced by **Virtual Memory (Paging)**.

Swapping

- **What it is:** The process of temporarily moving an entire process from main memory (RAM) to secondary storage (disk/SSD, often called **swap space**) and then bringing it back to RAM later.
- **Mechanism:** The OS identifies an inactive process, copies its entire memory image to the disk (**swap-out**), and frees up its RAM space for another process. When the original process is needed, it is copied back (**swap-in**), often to a different physical location, requiring **Execution Time Address Binding** to work.
- **Benefit:** Increases the degree of **multiprogramming** (number of programs running concurrently) by providing more temporary room in RAM.

Numerical Solving: Address Space Calculation

The problem provides the following parameters for a paged memory system:

- **Logical Address Space Size:** 8 Pages
- **Page Size (Block Size):** 1024 Words
- **Physical Memory Size:** 32 Frames
- **Frame Size:** Equal to Page Size (1024 Words)

In a paged system, an address is divided into two fields:

1. **Page/Frame Number (P/F):** Identifies the page in logical space or the frame in physical space.
2. **Offset/Displacement (D):** Identifies the specific word/byte within the page or frame.

$$\text{Address Size (in bits)} = \text{Bits for Page/Frame Number} + \text{Bits for Offset}$$

1. How Many Bits Are In Logical Address?

A **Logical Address (LA)** is used to access the logical address space. It requires bits to specify the **Page Number (P)** and the **Offset (D)** within that page.

A. Bits for Offset (D)

The offset field tells us the location within a page (or frame). The maximum number of bits needed to represent the offset is determined by the **Page Size**.

- Page Size = 1024 words
- We use the formula: $2^{\text{Bits for Offset}} = \text{Size}$
- $2^D = 1024$
- $1024 = 2^{10}$

$$\text{Bits for Offset (D)} = 10 \text{ bits}$$

B. Bits for Page Number (P)

The page number field tells us which of the total logical pages we are referring to. This is determined by the total number of pages in the logical space.

- Number of Pages = 8
- We use the formula: $2^{\text{Bits for Page No.}} = \text{Number of Pages}$
- $2^P = 8$
- $8 = 2^3$

$$\text{Bits for Page Number (P)} = 3 \text{ bits}$$

C. Total Logical Address Bits

The total size of the logical address is the sum of the page bits and the offset bits.

$$\text{Logical Address Bits} = P + D = 3 + 10$$

The Logical Address requires ****13 bits****

♾

2. How Many Bits Are In Physical Address?

A **Physical Address (PA)** is used to access the physical memory (RAM). It requires bits to specify the **Frame Number (F)** and the **Offset (D)** within that frame.

A. Bits for Offset (D)

Since frames are the same size as pages, the offset remains the same.

- Frame Size = 1024 words
- $2^D = 1024$

$$\text{Bits for Offset (D)} = 10 \text{ bits}$$

B. Bits for Frame Number (F)

The frame number field tells us which of the total physical frames we are referring to. This is determined by the total number of frames in the physical memory.

- Number of Frames = 32
- We use the formula: $2^{\text{Bits for Frame No.}} = \text{Number of Frames}$
- $2^F = 32$
- $32 = 2^5$

$$\text{Bits for Frame Number (F)} = 5 \text{ bits}$$

C. Total Physical Address Bits

The total size of the physical address is the sum of the frame bits and the offset bits.

$$\text{Physical Address Bits} = F + D = 5 + 10$$

The Physical Address requires ****15 bits****

✍

Summary Table

Address Component	Calculation	Bits
Page/Frame Offset (D)	$\log_2(1024)$	10

Address Component	Calculation	Bits
Logical Address	$\log_2(8) + D$	13
Physical Address	$\log_2(32) + D$	15

Holes In Memory Partitioning

- OS Keeps A Table Indicating Which Part Of Mem Are Available and which are occupied
- All Mem Is Available For User Process and Is Considered One Large Block Of Available Memory Known As Hole
- When Part Of Mem Is Occupied By A Process and Left Rest Of The Memory Also Known As Hole.
- A Hole Can Be Created When A Process Is Completed And Leaves The Memory

Memory Management is essential for efficiently sharing the computer's main memory (RAM). The techniques used to allocate this memory are broadly classified into two categories: **Contiguous** and **Non-Contiguous**.



Memory Management Techniques

1. Contiguous

In **Contiguous Memory Management**, every process must be loaded entirely and occupy a single, unbroken block of main memory (RAM). For a program requiring N bytes, the OS must find one available block of size N or greater.

2. Non-Contiguous

In **Non-Contiguous Memory Management** (like Paging and Segmentation), a process can be divided into several smaller pieces, and these pieces can be loaded into non-adjacent (separate) free blocks of memory. This allows for much more flexible and efficient use of fragmented RAM.



Contiguous Memory Management Techniques

Contiguous techniques were historically important and are simpler to implement but suffer from fragmentation issues.

1. Fixed / Static Partitioning (MFT: Multiprogramming with Fixed Tasks)

- **Definition:** The main memory is partitioned into a set of fixed-size, non-overlapping memory regions (or blocks) called **partitions** when the system is first configured (hence, "static").
- **Characteristics:**
 - Partitions can be of **equal size** or **unequal size**.
 - Each partition can hold exactly one process at a time.
 - The degree of multiprogramming (the number of processes that can run simultaneously) is fixed by the number of partitions.
- **Drawback:** It inevitably leads to **Internal Fragmentation**.

2. Variable / Dynamic Partitioning (MVT: Multiprogramming with Variable Tasks)

- **Definition:** Initially, the RAM is empty. Partitions are **not** created until a process arrives. During run time, the OS allocates a contiguous block of memory whose size is **equal to the incoming process's need**.
- **Characteristics:**

- The size of partitions varies according to the need of the process, ensuring the allocated space is just the right size.
- This technique **avoids internal fragmentation** because the allocated block size exactly matches the process size.
- **Drawback:** It inevitably leads to **External Fragmentation**.



Dynamic Memory Allocation Techniques

When using **Variable / Dynamic Partitioning**, the OS has a list of free memory regions, often called **holes**. When a new process arrives, the OS must decide which free hole to use. **First Fit**, **Best Fit**, and **Worst Fit** are the most common strategies used to select a free hole.

1. First Fit

- **Approach:** Allocate the **first free partition or hole** that is large enough to accommodate the process.
- **Process:** The search starts from the beginning of the free list and stops immediately after finding the first suitable hole.
- **Pros:** It is the **fastest** technique because the OS doesn't have to search the entire list.
- **Cons:** It tends to leave large holes at the beginning of memory, quickly filling the smaller holes and preventing larger future processes from finding space.

2. Best Fit

- **Approach:** Allocate the **smallest free partition** (hole) that is large enough to meet the requirements of the requesting process.
- **Process:** The OS must search the **entire list** of free partitions to find the hole whose size is closest to the actual process size needed.

- **Pros:** It conserves the largest free holes, leaving them available for larger processes later.
- **Cons:** It is the **slowest** due to the full list search and tends to leave many very small, unusable fragments (holes) in memory, which contributes heavily to **external fragmentation**.

3. Worst Fit

- **Approach:** Allocate the **largest free partition** (hole).
- **Process:** This is the reverse of Best Fit. The OS searches the entire list for the largest hole.
- **Goal:** The portion left over (the remainder) will be big enough to be useful for a smaller process later, hoping to reduce the number of tiny, unusable fragments.
- **Cons:** It quickly breaks up the largest contiguous blocks of memory, which might be critical for future large processes. This technique often performs poorly.

»

Fragmentation

Fragmentation is a critical concept in memory management and represents wasted space that cannot be used by processes, leading to inefficient RAM utilization.

Internal Fragmentation

- **Definition:** Wasted memory space that occurs **within a partitioned block of memory** allocated to a process.
- **How it Happens:** It is a problem specific to **Fixed / Static Partitioning**. If a partition size is 10 MB and a process only needs 6 MB, the remaining 4 MB is wasted, as it's allocated to the process but unused. It cannot be used by any other process.
- **Location:** The unused space is **inside** the allocated area.

- **Analogy:** A large T-shirt bought for a small child. The extra cloth is part of the shirt (allocated) but unused (wasted).

External Fragmentation

- **Definition:** Wasted memory space that occurs when there is **enough total free memory** to satisfy a process's request, but the free memory is **not contiguous** (it is split into many small, non-adjacent holes).
- **How it Happens:** It is a problem specific to **Variable / Dynamic Partitioning**. As processes are loaded and unloaded, the free space becomes scattered. If a process needs 10 MB, but the largest available contiguous hole is only 8 MB, the process cannot run, even if there are 50 MB of total free memory scattered around.
- **Location:** The unused space is **outside** any allocated area (the holes).
- **Analogy:** You have 100 empty parking spots, but they are all separated by 10 occupied cars. You cannot park a semi-truck (a large process) because you can't find 100 **adjacent** spots.



Numerical On Dynamic Memory Allocation

Solving Table

Process	Memory

Memory Partition	Process Number	Memory Requested	Status	Internal Fragmentation